```
getLogger(MySpringApp.class);

  public static void main(String[] args) {
    SpringApplication.run(MySpringApp.class, args);
    logger.info("Spring application started!");
  }
}
```

The assistant does not wash its hands off with just code. It gives a lot of notes as well. For instance, the following is the note generated along with the above code.

```
In this example, we use the LoggerFactory class from slf4j
to create a logger for the MySpringApp class. We can then
use the logger object to output log statements at different
levels (e.g., info, debug, error).
To use slf4j in a Spring application, you will need
to include the slf4j-api library as a dependency in
your project. You will also need to include a concrete
implementation of slf4j, such as log4j, logback, or java.
util.logging.
For example, to use log4j as the underlying logging
implementation, you will need to include the log4j library
as a dependency in your project. You can then configure log4j
to output log statements to the desired destination (e.g.,
console, file).
```

Since it is advising some dependencies, I acted as if I do not know how it can be done. So, I asked this follow-up question:

```
pom file for spring with slf4j
```

Here is the resulting suggestion:

```
<project>
  <!-- Other project configuration -->

  <dependencies>
    <!-- Other dependencies -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>1.7.30</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-log4j12</artifactId>
      <version>1.7.30</version>
    </dependency>
  </dependencies>
</project>
```

Again, the notes suggested a log4j dependency. I asked the assistant to add that too, and it generated the following refined configuration.

```
<project>
  <!-- Other project configuration -->

  <dependencies>
    <!-- Other dependencies -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>1.7.30</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-log4j12</artifactId>
      <version>1.7.30</version>
    </dependency>
    <dependency>
      <groupId>log4j</groupId>
      <artifactId>log4j</artifactId>
      <version>1.2.17</version>
    </dependency>
  </dependencies>
</project>
You will also need to include a log4j.properties file in your
project to configure log4j to output log statements to the
desired destination (e.g., console, file).
```

The story doesn't end here. I asked the assistant to help me with the *log4j.properties* file:

```
log4j.properties for spring with slf4j
```

It obliged.

```
# Root logger
log4j.rootLogger=INFO, stdout, file

# Console appender
log4j.appender.stdout=org.apache.log4j.ConsoleAppender
log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern=%d{yyyy-MM-dd
HH:mm:ss} %-5p %c{1}:%L - %m%n

# File appender
log4j.appender.file=org.apache.log4j.FileAppender
log4j.appender.file.File=app.log
log4j.appender.file.layout=org.apache.log4j.PatternLayout
log4j.appender.file.layout.ConversionPattern=%d{yyyy-MM-dd
HH:mm:ss} %-5p %c{1}:%L - %m%n
```